

Question Index - All 80 Questions

Q#	Question
Q1	Python basics for DevOps - why Python, use cases
Q2	Python data types - strings, lists, dicts, tuples, sets
Q3	Python file handling - read, write, context managers
Q4	Python string manipulation - f-strings, split, regex
Q5	Python error handling - try/except/finally
Q6	Python functions - args, kwargs, lambda, decorators
Q7	Python modules - import, pip, virtualenv, requirements.txt
Q8	Python OS interaction - os, subprocess, shutil, pathlib
Q9	Python JSON handling - json.load, json.dump, API parsing
Q10	Python HTTP requests - requests library, GET/POST
Q11	Python Boto3 basics - AWS SDK, EC2, S3, IAM
Q12	Python logging - levels, formatters, handlers
Q13	Groovy basics - syntax, variables, closures, vs Java
Q14	Groovy collections - lists, maps, ranges, GString
Q15	Groovy in Jenkins - scripted vs declarative pipelines
Q16	Groovy pipeline syntax - node, stage, steps, parallel
Q17	Groovy shared libraries - src/, vars/, resources/
Q18	Groovy string manipulation - regex, templates
Q19	Groovy file and HTTP - readFile, httpRequest, readJSON
Q20	Groovy closures and DSLs - Jenkins pipeline DSL
Q21	YAML basics - syntax, indentation, scalars, sequences
Q22	YAML anchors and aliases - DRY with ampersand and star

Q23	YAML multiline strings - literal pipe, folded
Q24	YAML gotchas - Norway problem, boolean coercion
Q25	YAML in DevOps - K8s, Compose, Ansible, CI configs
Q26	Python OOP - classes, inheritance, dataclasses
Q27	Python decorators deep dive - writing decorators
Q28	Python generators - yield, memory-efficient processing
Q29	Python context managers - custom with statements
Q30	Python async/await - asyncio, aiohttp, concurrency
Q31	Python CLI tools - argparse, click, DevOps scripts
Q32	Python Boto3 advanced - pagination, waiters, errors
Q33	Python Kubernetes client - kubernetes library
Q34	Python Docker SDK - managing containers programmatically
Q35	Python Jinja2 - templating config files
Q36	Python testing - pytest, mocking DevOps scripts
Q37	Python packaging - setup.py, pyproject.toml
Q38	Python virtual environments - venv, poetry, pipenv
Q39	Python performance - profiling, cProfile, memory
Q40	Python security - input validation, secrets, bandit
Q41	Groovy shared library patterns - reusable steps
Q42	Groovy pipeline error handling - try/catch, post
Q43	Groovy sandbox and security - script approval
Q44	Groovy testing - Spock framework, shared libs
Q45	Groovy pipeline optimization - parallel, matrix
Q46	JSON basics - syntax, data types, vs YAML

Q47	JSON Schema - validation, required fields, types
Q48	jq command-line - filters, transformations
Q49	jq advanced - select, map, reduce, scripting
Q50	HCL basics - blocks, attributes, expressions
Q51	HCL expressions - variables, locals, functions, loops
Q52	HCL vs JSON vs YAML - comparison
Q53	YAML validation - yamllint, schema, CI integration
Q54	JSON in CI/CD - parsing outputs, API responses
Q55	Config languages comparison - YAML vs JSON vs HCL vs TOML
Q56	Python infrastructure automation - custom providers
Q57	Python API development - FastAPI/Flask internal tools
Q58	Python data pipelines - processing logs, metrics, ETL
Q59	Groovy to declarative migration - scripted to declarative
Q60	Pipeline as code architecture - monorepo vs shared lib
Q61	YAML templating engines - ytt, Jsonnet, Dhall, Cue
Q62	Config anti-patterns - YAML hell, copy-paste, env drift
Q63	Polyglot scripting - when Python vs Bash vs Go vs Groovy
Q64	Internal developer platforms - CLI tools and SDKs
Q65	Code quality for scripts - linting, testing, CI
Q66	Python ModuleNotFoundError in CI but works locally
Q67	Boto3 script hangs - region, credential chain
Q68	Python OOM processing 10GB log file
Q69	Jenkins Groovy sandbox violation
Q70	Jenkins shared library breaking after upgrade

Q71	YAML indentation error breaks K8s deployment
Q72	YAML boolean coercion - yes becomes true
Q73	jq returns empty on nested JSON
Q74	Python requests timeout - retry logic
Q75	HCL for_each unexpected resource names
Q76	Groovy CPS transformation error
Q77	Python subprocess hangs - deadlock on pipes
Q78	YAML merge key deprecated - migration
Q79	Write Python: auto-remediate AWS SG violations
Q80	Write Groovy: reusable deployment pipeline

SECTION 1 - Junior Core Scripting (Q1–Q25)

Q1 Python basics for DevOps - why Python

The Simple Answer

Python is the most widely used scripting language in DevOps. It automates infrastructure management, integrates with every cloud provider (AWS Boto3, GCP google-cloud, Azure SDK), processes data (logs, metrics, configs), and builds internal tools. Every major DevOps tool either is written in Python (Ansible, SaltStack, OpenStack) or has a Python SDK.

The Analogy - A Swiss Army Knife

Bash is a screwdriver - great for quick, simple tasks on Linux. Python is a Swiss Army knife - it handles everything from file processing to API calls to cloud automation to building web dashboards. When a Bash script grows beyond 50 lines, reaches for arrays/dictionaries, or needs error handling, it is time to switch to Python.

Python vs Bash - When to Use Which

Use Bash: Simple file operations, piping commands, one-liners, system administration tasks under 50 lines, tasks that primarily chain Unix commands. Use Python: Anything over 50 lines, API interactions, JSON/YAML processing, cloud SDK operations, error handling with retry logic, tasks needing data structures (dicts, lists, classes), cross-platform scripts, or anything that will be maintained by a team.

****The rule of thumb:**** If you are writing if-else chains in Bash, parsing JSON with sed/awk, or building retry logic with sleep loops - switch to Python. Your future self will thank you.

Q2 Python data types

The Simple Answer

Strings - Text data. `name = "nginx"`. Use f-strings for interpolation: `f"Service {name} is running on port {port}"`. Immutable.

Lists - Ordered, mutable collections. `servers = ["web1", "web2", "web3"]`. Append, remove, sort, filter. Access by index: `servers[0]`. Slice: `servers[1:3]`.

Dictionaries - Key-value pairs. `config = {"port": 8080, "host": "0.0.0.0", "debug": False}`. Access: `config["port"]`. Get with default: `config.get("timeout", 30)`. The workhorse of DevOps scripting - every API response, config file, and cloud resource is a dictionary.

Tuples - Immutable ordered collections. Used for: function return values, dictionary keys, data that should not change. `coordinates = (40.7128, -74.0060)`.

Sets - Unordered unique elements. Used for: deduplication, membership testing, set operations (union, intersection, difference). `active_regions = {"us-east-1", "eu-west-1"}`.

DevOps Context

Parsing a Kubernetes YAML file produces nested dictionaries and lists. AWS API responses are dictionaries with lists of resources. Log processing involves string manipulation on each line. Server inventories are lists of dictionaries. Understanding these types and their operations is the foundation of all Python DevOps scripting.

Q3 Python file handling

The Simple Answer

File operations are fundamental to DevOps - reading configs, processing logs, generating reports, modifying templates.

Context Managers (the only correct way)

with `open("/var/log/app.log", "r")` as `f`: `lines = f.readlines()`. The with statement guarantees the file is closed even if an exception occurs. Never use `f = open(...)` without with - unclosed files cause resource leaks.

Modes

"r" - Read (default). "w" - Write (overwrites existing). "a" - Append. "r+" - Read and write. "b" suffix - Binary mode: "rb", "wb".

Common Patterns

Read entire file: with `open("config.yaml")` as `f`: `content = f.read()`. Read line by line (memory efficient for large files): with `open("huge.log")` as `f`: `for line in f: process(line)`. Write output: with `open("report.txt", "w")` as `f`: `f.write(f"Total servers: {count}\n")`. Process CSV: `import csv`; with `open("inventory.csv")` as `f`: `reader = csv.DictReader(f)`; `for row in reader: print(row["hostname"])`.

pathlib (Modern File Operations)

`from pathlib import Path`. `Path("/etc/nginx").exists()`. `Path("/tmp/output").mkdir(parents=True, exist_ok=True)`. `Path("config.json").read_text()`. More readable than `os.path`. The modern Python way.

Q4 Python string manipulation - f-strings, regex

f-strings (Python 3.6+)

The standard for string formatting: `name = "web-1"; port = 8080; url = f"http://{name}:{port}/health"`. Expressions inside braces: `f"Memory: {mem_mb / 1024:.2f} GB"`. The fastest and most readable string formatting method.

Essential String Methods

"hello ".strip() - Remove whitespace. "web-1,web-2,web-3".split(",") - Split into list. ", ".join(["a", "b", "c"]) - Join list into string. "nginx.conf".endswith(".conf") - Check suffix. "ERROR: disk full".startswith("ERROR") - Check prefix. "Hello World".lower() - Lowercase. line.replace("old", "new") - Replace substring.

Regular Expressions (re module)

import re. re.search(r"ERROR (\d+)", log_line) - Find a pattern, capture groups. re.findall(r"\d+\.\d+\.\d+\.\d+", text) - Find all IP addresses. re.sub(r"password=\S+", "password=***", config) - Redact passwords. re.match(r"^(\w+)\s+(\d+)", line) - Match at the start.

DevOps Use Cases

Parse log files for error patterns. Extract IP addresses from firewall rules. Redact sensitive data before logging. Validate configuration values (port numbers, CIDR ranges, hostnames). Generate dynamic configuration from templates.

Q5 Python error handling - try/except/finally

The Simple Answer

DevOps scripts interact with unreliable systems - networks drop, APIs timeout, files are missing, permissions are denied. Error handling is not optional - it is the difference between a script that crashes silently at 3 AM and one that retries, logs the error, and alerts the team.

The Pattern

```
try: response = requests.get(url, timeout=10). except requests.Timeout: logger.error(f"Timeout connecting to {url}"). except requests.ConnectionError: logger.error(f"Cannot reach {url}"). except Exception as e: logger.error(f"Unexpected error: {e}"). finally: cleanup_resources().
```

The Analogy

try/except is like a safety net for a trapeze artist. The try block is the performance. The except block is the net - it catches specific types of falls. The finally block is the cleanup crew - they run regardless of whether the performance succeeded or the artist fell. Without a safety net, one mistake ends the show (and wakes up the on-call engineer).

Best Practices

Catch specific exceptions, not bare except:. Log the error with context (which server, which operation, what input). Implement retry logic for transient failures (network timeouts). Use finally for cleanup (close connections, release locks, delete temp files). Raise custom exceptions for domain-specific errors: class DeploymentError(Exception): pass.

Q6 Python functions - args, kwargs, lambda

The Simple Answer

Functions organize code into reusable units. In DevOps scripts, functions encapsulate: API calls, retry logic, config parsing, resource management.

args and kwargs

*args - Variable positional arguments: `def deploy(*services): for s in services: deploy_one(s)`. Call: `deploy("web", "api", "worker")`. kwargs - Variable keyword arguments: `def create_instance(params):` used for passing dynamic configuration. Call: `create_instance(type="t3.medium", ami="ami-123", tags={"env": "prod"})`.

Lambda Functions

Short anonymous functions: `sorted.instances, key=lambda i: i["launch_time"]`). Sort instances by launch time. `filter(lambda i: i["state"] == "running", instances)`. Filter to running instances only. Use for: simple transformations in sort, filter, map. If the lambda is complex, use a regular function instead.

Default Arguments

`def connect(host, port=5432, timeout=30, ssl=True):` practical for configuration with sensible defaults. Callers override only what they need: `connect("db.example.com")` uses all defaults. `connect("db.example.com", ssl=False)` overrides one parameter.

Q7 Python modules - import, pip, virtualenv

The Simple Answer

Modules are Python files that you import. Packages are directories of modules. pip installs third-party packages. Virtual environments isolate project dependencies.

Essential DevOps Packages

boto3 - AWS SDK. requests - HTTP calls. pyyaml - YAML parsing. jinja2 - Template rendering. paramiko - SSH connections. docker - Docker SDK. kubernetes - K8s client. click - CLI frameworks. pytest - Testing. rich - Beautiful terminal output.

requirements.txt

Pin versions: `boto3==1.28.0, requests>=2.28.0,<3.0.0`. Generate: `pip freeze > requirements.txt`. Install: `pip install -r requirements.txt`. Always pin in production scripts - unpinned dependencies break randomly when a new version is released.

Virtual Environments

`python3 -m venv .venv` creates an isolated environment. `source .venv/bin/activate` activates it. pip install only affects this environment. Different projects can use different versions of the same package. Always use virtual environments - never pip install globally.

Q8 Python OS interaction - os, subprocess, pathlib

subprocess (Running Commands)

`result = subprocess.run(["kubectl", "get", "pods", "-o", "json"], capture_output=True, text=True, check=True)`. `capture_output=True` captures stdout and stderr. `text=True` returns strings (not bytes). `check=True` raises `CalledProcessError` on non-zero exit code. Access output: `result.stdout`, `result.returncode`.

os Module

`os.environ["AWS_REGION"]` - Read environment variables. `os.getenv("DB_PASSWORD", "default")` - With default. `os.path.exists("/etc/nginx/nginx.conf")` - Check file exists. `os.listdir("/var/log/")` - List directory contents. `os.makedirs("/opt/app/config", exist_ok=True)` - Create directories.

shutil (File Operations)

`shutil.copy("src.conf", "/etc/app/app.conf")` - Copy file. `shutil.copytree("templates/", "/opt/app/templates/")` - Copy directory. `shutil.rmtree("/tmp/build/")` - Remove directory tree.

****subprocess vs os.system:**** Never use `os.system()` - it does not capture output, does not raise on errors, and is vulnerable to shell injection. Always use `subprocess.run()` with a list of arguments (not a shell string).

Q9 Python JSON handling

The Simple Answer

JSON is the lingua franca of APIs and configuration. Every cloud API, every REST endpoint, and every Kubernetes resource returns JSON. Python's `json` module is essential for DevOps.

Parsing JSON

`import json`. `data = json.loads('{"name": "web-1", "port": 8080}')` - Parse JSON string to dict. with `open("config.json") as f: config = json.load(f)` - Parse JSON file. Access nested data: `config["database"]["host"]`, `config["servers"][0]["ip"]`.

Generating JSON

`json.dumps(data, indent=2)` - Dict to formatted JSON string. with `open("output.json", "w") as f: json.dump(results, f, indent=2)` - Write to file.

Real-World Pattern

`response = requests.get("https://api.example.com/servers")`. `servers = response.json()` - Parse API response directly. `for server in servers: print(f'{server["name"]}: {server["status"]}')`. This pattern (call API → parse JSON → process data) is the foundation of cloud automation scripts.

Q10 Python HTTP requests

The Simple Answer

The requests library is Python's standard for HTTP operations. Every DevOps automation interacts with APIs - cloud providers, monitoring systems, CI/CD tools, internal services.

Essential Operations

GET: `response = requests.get("https://api.example.com/status", headers={"Authorization": f"Bearer {token}"}, timeout=10)`. POST: `response = requests.post(url, json={"name": "web-3", "type": "t3.medium"}, timeout=10)`. PUT/PATCH/DELETE: Same pattern with `requests.put()`, `requests.patch()`, `requests.delete()`.

Response Handling

`response.status_code` - HTTP status (200, 404, 500). `response.json()` - Parse JSON body. `response.text` - Raw text body. `response.raise_for_status()` - Raise `HTTPError` for 4xx/5xx responses. `response.headers` - Response headers.

Authentication Patterns

Bearer token: `headers={"Authorization": f"Bearer {token}"}`. Basic auth: `requests.get(url, auth=("user", "password"))`. API key: `headers={"X-API-Key": api_key}` or `params={"api_key": key}`.

****Always set timeout:**** `requests.get(url, timeout=10)`. Without timeout, the request hangs indefinitely if the server is unresponsive. Your script freezes. Your pipeline hangs. See scenario Q74.

Q11 Python Boto3 basics - AWS SDK

The Simple Answer

Boto3 is the AWS SDK for Python. It provides programmatic access to every AWS service - EC2, S3, IAM, Lambda, RDS, ECS, and 200+ more. Most AWS automation scripts use Boto3.

Client vs Resource

Client (low-level): Direct API calls. `ec2_client = boto3.client("ec2", region_name="us-east-1")`. Response is raw JSON dict. More control, more verbose. Resource (high-level): Object-oriented. `s3 = boto3.resource("s3"). bucket = s3.Bucket("my-bucket")`. Cleaner code but not available for all services.

Common Operations

List EC2 instances: `ec2.describe_instances(Filters=[{"Name": "instance-state-name", "Values": ["running"]}])`. Upload to S3: `s3.upload_file("report.pdf", "my-bucket", "reports/2025/report.pdf")`. Create IAM user: `iam.create_user(UserName="deploy-bot")`. Invoke Lambda: `lambda_client.invoke(FunctionName="process-logs", Payload=json.dumps(event))`.

Credential Chain

Boto3 looks for credentials in order: environment variables (`AWS_ACCESS_KEY_ID`) → AWS config file (`~/.aws/credentials`) → IAM instance profile (EC2 role) → ECS task role → EKS service account. Best

practice: never hardcode credentials. Use IAM roles for EC2/ECS/Lambda. Use environment variables or AWS SSO for local development.

Q12 Python logging

The Simple Answer

`print()` is for debugging. logging is for production scripts. The logging module provides: severity levels, timestamps, output to files and streams, and structured formatting.

Setup

```
import logging
logging.basicConfig(level=logging.INFO, format="%(asctime)s [%(levelname)s]
%(message)s", datefmt="%Y-%m-%d %H:%M:%S")
logger = logging.getLogger(__name__)
```

Levels

`logger.debug("Variable x = 42")` - Detailed debugging. `logger.info("Deployment started for version 2.1.0")` - Normal operations. `logger.warning("Disk usage at 85%")` - Potential issue. `logger.error("Failed to connect to database")` - Error occurred. `logger.critical("All replicas down - service unavailable")` - System-level failure.

DevOps Best Practices

Log at INFO level for normal operations. Include context: `logger.error(f"Failed to deploy {service} to {environment}: {error}")`. Use structured logging (JSON format) for log aggregation tools (ELK, Loki): `import json; logger.info(json.dumps({"event": "deploy", "service": "api", "version": "2.1.0", "status": "success"}))`. Never log secrets - redact passwords, tokens, and keys.

Q13 Groovy basics - syntax, closures

The Simple Answer

Groovy is a JVM language that is the scripting language for Jenkins pipelines. If you use Jenkins, you write Groovy. It is like Java but with less boilerplate - optional semicolons, optional type declarations, built-in closures, and powerful string handling.

The Analogy

If Java is writing a formal business letter (structured, verbose, strict), Groovy is writing an email (same language, shorter, more flexible). Java: `String name = new String("hello");` Groovy: `def name = "hello"`. Same JVM, less ceremony.

Key Syntax

Variables: `def name = "nginx"` (dynamic typing) or `String name = "nginx"` (static). Closures: `def greet = { name -> println "Hello, ${name}" }; greet("world")`. Closures are code blocks you can pass around - the

foundation of Jenkins pipeline DSL. String interpolation: `def port = 8080; def url = "http://localhost:${port}"`. Note: double quotes for interpolation, single quotes for literal strings.

Q14 Groovy collections - lists, maps

The Simple Answer

Lists: `def servers = ["web1", "web2", "web3"]; servers.each { println it }; servers.findAll { it.startsWith("web") }; servers.collect { it.toUpperCase() }.`

Maps: `def config = [port: 8080, host: "0.0.0.0", debug: false]; config.port` returns 8080. `config.each { key, value -> println "${key}: ${value}" }.`

GString interpolation: `def name = "app"; def result = "Deploying ${name} to ${config.host}:${config.port}"`. GStrings evaluate expressions inside `${}` - including method calls and ternary operators.

Q15 Groovy in Jenkins - scripted vs declarative

Scripted Pipeline

`node { stage("Build") { sh "mvn clean package" }; stage("Test") { sh "mvn test" }; stage("Deploy") { sh "./deploy.sh" } }`. Full Groovy power. Any Groovy code is valid. Flexible but complex. Harder to read for non-programmers.

Declarative Pipeline

`pipeline { agent any; stages { stage("Build") { steps { sh "mvn clean package" } }; stage("Test") { steps { sh "mvn test" } } }; post { failure { slackSend "Build failed" } } }`. Structured, opinionated. Easier to read and maintain. Supports: when conditions, parallel stages, matrix builds, input prompts. The recommended approach for new pipelines.

****The evolution:**** Jenkins started with Freestyle jobs (GUI configuration). Scripted pipelines added code-based configuration. Declarative pipelines added structure and guardrails. New pipelines should always be declarative. Use `script { }` blocks within declarative for the rare cases needing full Groovy power.

Q16 Groovy pipeline syntax - stage, steps, parallel

Stage and Steps

Stages organize the pipeline visually. Each stage contains steps - the actual commands. `stage("Build") { steps { sh "docker build -t myapp:${BUILD_NUMBER}" } }`. `stage("Test") { steps { sh "docker run myapp:${BUILD_NUMBER} npm test" } }`.

Parallel Execution

stage("Tests") { parallel { stage("Unit") { steps { sh "npm test" } }; stage("Integration") { steps { sh "npm run test:integration" } }; stage("Lint") { steps { sh "npm run lint" } } } }. All three test types run simultaneously - pipeline completes when the slowest finishes.

Post Actions

post { always { junit "target/test-results/*.xml" }; success { slackSend "Build passed" }; failure { slackSend "Build FAILED" }; cleanup { sh "docker image prune -f" } }. Always runs regardless of outcome. Success/failure runs conditionally. Cleanup runs last.

Q17 Groovy shared libraries

The Simple Answer

Shared libraries are reusable Groovy code stored in a Git repository and loaded by Jenkins pipelines. Instead of copying the same deployment logic into 50 Jenkinsfiles, define it once in a shared library and call it from every pipeline.

Directory Structure

vars/ - Global pipeline steps. vars/deployService.groovy defines a deployService() step callable from any pipeline. Each file becomes a pipeline step: deployService(service: "api", version: "2.1.0"). src/ - Groovy classes. Object-oriented code for complex logic. Import and use in pipelines. resources/ - Static files. Templates, scripts, configuration files loaded with libraryResource().

Loading

In Jenkinsfile: @Library("my-shared-lib") _ . Or in Jenkins global configuration: auto-load for all pipelines. The library is cloned from Git at pipeline start.

Q18 Groovy string manipulation

The Simple Answer

GString interpolation: def branch = "feature/login"; def msg = "Building branch: \${branch}". Regex matching: def version = "v2.1.0"; if (version =~ /v\d+\.\d+\.\d+/) { echo "Valid version" }. Extract with regex: def match = ("Release-2.1.0" =~ /Release-(\d+\.\d+\.\d+)/); def ver = match[0][1]. Multiline strings: def script = """ docker build -t app:\${version} . docker push app:\${version} """.

Q19 Groovy file and HTTP in Jenkins

The Simple Answer

readFile: def config = readFile("config.yaml"). Reads a file from the workspace. writeFile: writeFile(file: "output.txt", text: "Deploy complete"). readJSON: def data = readJSON(file: "package.json"); echo data.version. readYaml: def manifest = readYaml(file: "deployment.yaml"). httpRequest: def response = httpRequest(url: "https://api.example.com/status", authentication: "api-creds-id"); echo response.content.

Q20 Groovy closures and DSLs

The Simple Answer

A closure is a block of code that can be assigned to a variable, passed as an argument, and executed later. Jenkins pipeline DSL is built entirely on closures.

How Jenkins Pipeline DSL Works

When you write: `stage("Build") { steps { sh "make" } } - stage()` is a function that takes a string and a closure. `steps` is a function that takes a closure. `sh` is a function inside the `steps` closure. The entire Jenkinsfile is nested closures - Groovy's closures make this clean DSL syntax possible.

Custom DSL Example

In a shared library: `def call(Map config, Closure body) { stage("Deploy ${config.service}") { body() } }`. Pipeline uses it as: `deployStage(service: "api") { sh "./deploy.sh" }`. This is how shared libraries create clean, domain-specific pipeline syntax.

Q21 YAML basics - syntax, indentation

The Simple Answer

YAML (YAML Ain't Markup Language) is the configuration language of DevOps. Kubernetes manifests, Docker Compose files, Ansible playbooks, GitHub Actions workflows, and Helm values files are all YAML.

The Analogy

JSON is like filling out a tax form - every bracket, comma, and quote must be perfect. YAML is like writing a neatly indented outline - the structure is defined by indentation, which is more human-readable but more error-prone (one wrong space breaks everything).

Syntax Rules

Indentation matters: Use 2 spaces (convention). NEVER use tabs - YAML rejects tabs. Key-value pairs: `name: nginx, port: 8080`. Lists (sequences): items starting with dash-space: `- web1, - web2`. Nested structures: indent by 2 spaces for nesting. Comments: `# This is a comment`. Null: `key: null` or `key: ~`.

Data Types

Strings: `name: "hello"` or `name: hello` (quotes optional for simple strings). Numbers: `port: 8080` (integer), `ratio: 0.95` (float). Booleans: `debug: true, verbose: false`. Note: YAML also treats `yes, no, on, off` as booleans - this causes bugs. See Q24.

Q22 YAML anchors and aliases

The Simple Answer

Anchors (&) and aliases (*) enable reuse within a YAML file - the DRY (Don't Repeat Yourself) principle applied to configuration.

Example

defaults: &defaults adapter: postgres host: localhost port: 5432. development: <<: *defaults database: myapp_dev. production: <<: *defaults database: myapp_prod host: prod-db.example.com. The << merge key inserts all key-value pairs from the anchor. Override specific values after the merge.

Limitations

Anchors and aliases work only within a SINGLE YAML file - not across files. The << merge key is technically not part of the YAML spec (it is a YAML 1.1 extension). Some tools do not support it. See scenario Q78 for deprecation issues.

Q23 YAML multiline strings

Literal Block (pipe |)

Preserves newlines exactly as written. content: | line one line two line three. Result: "line one\nline two\nline three\n". Use for: shell scripts in CI/CD configs, multi-line commands, preserving formatting.

Folded Block (>)

Folds newlines into spaces (like word wrap). description: > This is a long description that wraps across multiple lines. Result: "This is a long description that wraps across multiple lines\n". Use for: long strings that should be a single line in the output.

Chomp Indicators

|+ keeps trailing newlines. |- strips trailing newlines. | (default) keeps one trailing newline. Same for > + and > -.

Q24 YAML gotchas - Norway problem, booleans

The Norway Problem

In YAML 1.1, country codes are treated as booleans: NO (Norway) becomes false, YES becomes true. This breaks: country lists, configuration flags, and any value that looks like a boolean. Fix: always quote ambiguous values: country: "NO".

Boolean Coercion

YAML 1.1 treats these as booleans: yes, no, true, false, on, off, y, n (case-insensitive). debug: yes becomes debug: true. enabled: NO becomes enabled: false. This breaks configurations where "yes" should be a string. Fix: quote all boolean-like values: answer: "yes". YAML 1.2 (used by most modern parsers) restricts booleans to only true/false, but many tools still use YAML 1.1.

The Sexagesimal Trap

Unquoted values like 12:30 are interpreted as base-60 numbers (750), not strings. Port ranges like 22:80 become 1400. Fix: quote: time: "12:30", ports: "22:80".

The Octal Trap

Numbers starting with 0 are interpreted as octal in YAML 1.1: 0644 becomes 420 (decimal). File permissions must be quoted: mode: "0644".

****The universal YAML rule:**** When in doubt, quote it. Strings in quotes are always strings, never coerced. The 5 minutes spent quoting saves hours of debugging YAML-induced production incidents. See scenario Q72.

Q25 YAML in DevOps - K8s, Compose, Ansible

The Simple Answer

YAML is everywhere in DevOps. Each tool uses YAML with its own conventions.

Kubernetes - Manifests: apiVersion, kind, metadata, spec. Strict about types. Apply with kubectl apply -f.
Docker Compose - Services, networks, volumes. version field (deprecated in modern Compose). Relaxed parsing.
Ansible - Playbooks and inventory. Uses Jinja2 templating inside YAML ({{ variable }}).
GitHub Actions - Workflow files in .github/workflows/. Triggers, jobs, steps.
Helm values - values.yaml customizes chart behavior. Nested structures common.
GitLab CI - .gitlab-ci.yml defines pipeline stages and jobs. Anchors heavily used for DRY.

****The meta-skill:**** Reading and writing YAML fluently is a prerequisite for every DevOps role. Most DevOps interview coding tasks involve writing YAML - Kubernetes manifests, CI/CD pipelines, or Ansible playbooks. Practice writing YAML by hand, without auto-complete.

SECTION 2 - Mid-Level Production Patterns (Q26–Q55)

Q26 Python OOP - classes, dataclasses

The Simple Answer

Classes encapsulate related data and behavior. In DevOps scripts, classes model: cloud resources (EC2Instance, S3Bucket), deployment configurations (DeployConfig), and service clients (APIClient with retry logic, authentication, and logging built in).

dataclasses (Python 3.7+)

from dataclasses import dataclass. @dataclass class ServerConfig: hostname: str; port: int = 8080; ssl: bool = True; region: str = "us-east-1". Auto-generates __init__, __repr__, __eq__. Less boilerplate than regular classes. Use for: configuration objects, API response models, structured data.

When OOP Matters in DevOps

A 50-line script does not need classes. A 500-line automation tool does. Classes provide: encapsulation (API client hides auth details), reusability (one S3Client class used across 20 scripts), testability (mock the class, not individual functions), and maintainability (change retry logic in one place).

Q27 Python decorators deep dive

The Simple Answer

A decorator wraps a function with additional behavior without modifying the function itself. The most common DevOps use case: retry logic.

The Retry Decorator Pattern

```
import functools, time. def retry(max_attempts=3, delay=2): def decorator(func):
@functools.wraps(func) def wrapper(*args, kwargs): for attempt in range(max_attempts): try: return
func(*args, kwargs) except Exception as e: if attempt == max_attempts - 1: raise; time.sleep(delay *
(attempt + 1)); return wrapper; return decorator.
```

Usage: @retry(max_attempts=5, delay=3) def deploy_service(name, version): ... Every call to deploy_service automatically retries up to 5 times with increasing delays. The function itself has no retry logic - the decorator adds it transparently.

Other DevOps Decorator Patterns

@timer - Log execution time for performance monitoring. @log_call - Log function entry/exit with arguments. @require_env("AWS_REGION") - Verify environment variable exists before execution. @cached(ttl=300) - Cache expensive API call results for 5 minutes.

Q28 Python generators - yield

The Simple Answer

Generators produce values one at a time instead of creating an entire list in memory. Essential for processing large datasets - log files, API pagination, stream processing.

The Analogy

A list is like reading an entire book into memory before starting. A generator is like reading one page at a time - you only hold one page in memory, regardless of book size. For a 10 GB log file, a list requires 10 GB of RAM. A generator requires kilobytes.

The Pattern

```
def read_large_log(path): with open(path) as f: for line in f: if "ERROR" in line: yield line. Usage: for error_line in read_large_log("/var/log/app.log"): process(error_line). Only one line is in memory at a time - even for a 10 GB file.
```

API Pagination Generator

```
def get_all_instances(ec2_client): paginator = ec2_client.get_paginator("describe_instances"); for page in paginator.paginate(): for reservation in page["Reservations"]: for instance in reservation["Instances"]: yield instance. Handles AWS pagination transparently - callers get a clean stream of instances regardless of how many API pages exist.
```

Q29 Python context managers

The Simple Answer

Context managers guarantee cleanup with the with statement. Built-in: file handles (with open()), locks (with threading.Lock()), database connections. Custom context managers handle any resource that needs cleanup.

Writing Custom Context Managers

```
from contextlib import contextmanager. @contextmanager def temporary_directory(): import tempfile, shutil; tmpdir = tempfile.mkdtemp(); try: yield tmpdir; finally: shutil.rmtree(tmpdir). Usage: with temporary_directory() as tmpdir: download_files(tmpdir); process_files(tmpdir). The directory is guaranteed to be cleaned up even if an exception occurs.
```

DevOps Use Cases

SSH connection manager: with ssh_connect("web-1") as conn: conn.exec("systemctl restart nginx"). AWS session: with assumed_role("deploy-role") as session: deploy(session). Database transaction: with db.transaction() as tx: tx.execute(migration); tx.commit(). Each guarantees cleanup - connection closed, role released, transaction rolled back on error.

Q30 Python async/await - asyncio

The Simple Answer

Async allows concurrent operations without threads. When one operation waits for I/O (network response, file read), another operation runs. Essential when your script makes many independent API calls.

The Problem Async Solves

Checking health of 100 servers sequentially: 100 servers x 2 seconds timeout = 200 seconds. Checking concurrently with async: all 100 requests sent simultaneously, total time = ~2 seconds (slowest response).

The Pattern

```
import asyncio, aiohttp. async def check_health(session, url): async with session.get(url, timeout=5) as resp: return {"url": url, "status": resp.status}. async def check_all(urls): async with aiohttp.ClientSession() as session: tasks = [check_health(session, url) for url in urls]; return await asyncio.gather(*tasks). results = asyncio.run(check_all(server_urls)).
```

****When to use async:**** Many independent I/O operations (API calls, health checks, file downloads).
When to avoid: CPU-bound work (use multiprocessing instead), simple sequential scripts, or when the synchronous requests library is sufficient.

Q31 Python CLI tools - argparse, click

The Simple Answer

DevOps scripts need command-line interfaces. argparse is built-in. click is a popular library with a cleaner API and more features.

click Example

```
import click. @click.command(). @click.option("--service", required=True, help="Service to deploy"). @click.option("--version", required=True, help="Version tag"). @click.option("--env", type=click.Choice(["dev", "staging", "prod"]), default="dev"). @click.option("--dry-run", is_flag=True, help="Preview without deploying"). def deploy(service, version, env, dry_run): ...
```

Usage: python deploy.py --service api --version v2.1.0 --env staging --dry-run. click provides: type validation, help text, tab completion, colored output, confirmation prompts, and progress bars.

Q32 Python Boto3 advanced - pagination, waiters

Pagination

AWS APIs return maximum 100 results per call. For 500 instances, you need 5 API calls. Paginators handle this: paginator = ec2.get_paginator("describe_instances"); for page in paginator.paginate(Filters=[...]): for r in page["Reservations"]: for i in r["Instances"]: process(i). Never manually handle NextToken - always use paginators.

Waiters

Wait for a resource to reach a desired state: `waiter = ec2.get_waiter("instance_running"); waiter.wait(InstanceIds=["i-abc123"], WaiterConfig={"Delay": 15, "MaxAttempts": 40})`. Blocks until the instance is running (or timeout). Built-in waiters for: `instance_running`, `instance_stopped`, `snapshot_completed`, `stack_create_complete`.

Error Handling

`from boto3.exceptions import ClientError. try: ec2.describe_instances(...) except ClientError as e: if e.response["Error"]["Code"] == "InvalidInstanceID.NotFound": handle_missing() else: raise.` Always catch `ClientError` and check the error code - different errors need different handling.

Q33 Python Kubernetes client

The Simple Answer

The kubernetes Python library provides programmatic access to Kubernetes clusters. List pods, create deployments, watch events, and manage resources from Python scripts.

Common Operations

`from kubernetes import client, config. config.load_kube_config() (local) or config.load_incluster_config() (inside a Pod). v1 = client.CoreV1Api(). pods = v1.list_namespaced_pod("production"). for pod in pods.items: print(f"{pod.metadata.name}: {pod.status.phase}").`

DevOps Use Cases

Custom health checker: List pods, check readiness conditions, alert on unhealthy pods. Cleanup script: Delete completed jobs older than 24 hours. Scaling automation: Watch CPU metrics, scale deployments programmatically. Migration script: Update image tags across all deployments in a namespace.

Q34 Python Docker SDK

The Simple Answer

The docker Python library manages containers, images, networks, and volumes programmatically. Replaces docker CLI commands with Python API calls.

Common Operations

`import docker. client = docker.from_env(). containers = client.containers.list(). container = client.containers.run("nginx:latest", detach=True, ports={"80/tcp": 8080}). container.logs(). container.stop(). client.images.build(path=".", tag="myapp:latest"). client.images.push("registry.example.com/myapp:latest").`

Q35 Python Jinja2 - templating

The Simple Answer

Jinja2 is Python's templating engine. Used by Ansible, Flask, and custom DevOps tools to generate configuration files from templates.

Pattern

```
from jinja2 import Environment, FileSystemLoader. env =
Environment(loader=FileSystemLoader("templates/")). template = env.get_template("nginx.conf.j2").
output = template.render(server_name="api.example.com", upstream_servers=["10.0.1.5", "10.0.1.6"],
ssl_enabled=True). Path("nginx.conf").write_text(output).
```

DevOps Use Cases

Generate Kubernetes manifests with environment-specific values. Create Nginx/HAProxy configs from service discovery data. Build monitoring dashboards from templates. Generate Terraform backend configs per environment.

Q36 Python testing - pytest for DevOps scripts

The Simple Answer

DevOps scripts need tests too. A deploy script without tests is a ticking time bomb - one wrong variable and you deploy to production instead of staging.

Mocking AWS/Cloud APIs

```
from unittest.mock import patch, MagicMock. @patch("boto3.client") def
test_stop_instances(mock_client): mock_ec2 = MagicMock(); mock_client.return_value = mock_ec2;
mock_ec2.describe_instances.return_value = {"Reservations": [...]}; stop_idle_instances();
mock_ec2.stop_instances.assert_called_once_with(InstanceIds=["i-123"]).
```

Using moto (AWS Mock)

```
from moto import mock_s3. @mock_s3 def test_upload(): s3 = boto3.client("s3", region_name="us-
east-1"); s3.create_bucket(Bucket="test-bucket"); upload_report("test-bucket", "report.txt"); obj =
s3.get_object(Bucket="test-bucket", Key="report.txt"); assert obj["Body"].read() == b"report content".
moto simulates AWS services locally - no real AWS calls. The gold standard for testing Boto3 code.
```

Q37 Python packaging - pyproject.toml

The Simple Answer

When a DevOps script grows into a tool used by the team, package it properly. pyproject.toml (modern standard) replaces setup.py. Define: name, version, dependencies, entry points (CLI commands), and Python version requirements. pip install -e . installs in development mode. pip install . installs for production.

Q38 Python virtual environments - venv, poetry

The Simple Answer

venv: Built-in. python3 -m venv .venv && source .venv/bin/activate. Simple. No extra tools. Use for: single-project isolation.

poetry: Modern dependency manager. poetry init → poetry add boto3 → poetry install. Creates virtual environment automatically. Generates poetry.lock for exact reproducibility. Use for: projects with many dependencies, publishing packages.

pipenv: Combines pip + virtualenv. Pipfile and Pipfile.lock. Less popular than poetry in 2025.

****The rule:**** Always use virtual environments. Never pip install globally. Each project gets its own isolated environment. CI pipelines should create a fresh venv for each run.

Q39 Python performance - profiling

The Simple Answer

When a Python script is slow, profile it - do not guess. python -m cProfile script.py shows where time is spent. line_profiler shows per-line execution time. memory_profiler shows memory usage per line.

Common DevOps Performance Issues

Processing large files without streaming: Loading a 5 GB log file into memory with f.read() - use generators instead. Synchronous API calls in loops: 100 sequential HTTP requests - use async or ThreadPoolExecutor. String concatenation in loops: result += line is $O(n^2)$ - use ".join(lines)" instead. Not using built-in functions: Custom Python loops are 10x slower than built-in functions (map, filter, list comprehensions).

Q40 Python security - input validation, bandit

The Simple Answer

DevOps scripts handle sensitive data and execute system commands - security is critical.

Key Practices

Never use shell=True in subprocess. subprocess.run(f"rm -rf {user_input}", shell=True) is command injection. Use: subprocess.run(["rm", "-rf", validated_path]). Never use eval() or exec() with untrusted input. Use secrets module for tokens: import secrets; token = secrets.token_hex(32) - cryptographically secure. Validate all inputs: Verify CIDR ranges, hostnames, and port numbers before using them in commands. bandit - Python SAST: pip install bandit && bandit -r src/ scans for: hardcoded passwords, SQL injection, unsafe YAML loading (yaml.safe_load not yaml.load), and insecure subprocess calls.

Q41 Groovy shared library patterns

The Simple Answer

Mature shared libraries provide reusable pipeline steps that encapsulate organizational deployment patterns.

vars/ Pattern (Simple Steps)

```
vars/deployToKubernetes.groovy: def call(Map config) { stage("Deploy ${config.service}") {  
    container("kubectl") { sh "kubectl set image deployment/${config.service}  
    ${config.service}=${config.image}:${config.tag}" } } }. Pipeline uses: deployToKubernetes(service: "api",  
image: "registry/api", tag: env.BUILD_NUMBER).
```

src/ Pattern (Class-Based)

src/com/company/DeployPipeline.groovy: class with methods for build, test, deploy, notify. Pipelines instantiate: `def pipeline = new DeployPipeline(this); pipeline.standardDeploy(service: "api")`. More testable and reusable than vars/ functions.

Q42 Groovy pipeline error handling

The Simple Answer

Pipelines fail. Error handling determines whether a failure is: silently ignored (dangerous), properly logged and notified (correct), or automatically remediated (advanced).

Declarative Post Actions

```
post { always { archiveArtifacts "reports/**" }; success { slackSend "Deployed ${VERSION}" }; failure {  
    slackSend "FAILED: ${BUILD_URL}" }; unstable { slackSend "Tests flaky" }; cleanup { sh "docker system  
    prune -f" } }.
```

Script Block Try/Catch

```
stage("Deploy") { steps { script { try { sh "./deploy.sh" } catch (Exception e) { sh "./rollback.sh";  
    error("Deploy failed: ${e.message}") } } } }.
```

catchError

`catchError(buildResult: "SUCCESS", stageResult: "FAILURE") { sh "./optional-step.sh" }`. The stage fails but the pipeline continues. Used for: non-critical steps (upload coverage report) that should not block deployment.

Q43 Groovy sandbox and security

The Simple Answer

Jenkins runs Groovy in a sandbox that restricts dangerous operations. The sandbox blocks: file system access (`new File()`), network calls (`new URL().openConnection()`), system commands (`Runtime.exec()`), and Java reflection. This prevents malicious or accidental damage from pipeline scripts.

When the Sandbox Blocks You

"Scripts not permitted to use method `java.io.File`" - the sandbox rejected your code. Fix: use Jenkins pipeline steps instead of raw Groovy/Java. `readFile()` instead of `new File()`. `sh "curl ..."` instead of `URL.openConnection()`. `HttpRequest` instead of `URLConnection`. If no pipeline step exists, a Jenkins admin approves the method in Manage Jenkins → In-process Script Approval.

Q44 Groovy testing - Spock

The Simple Answer

Spock is the standard Groovy testing framework. BDD-style: given/when/then blocks. Shared library functions can be unit tested with Spock by mocking Jenkins pipeline steps.

Testing Shared Libraries

Mock the pipeline context: `def steps = Mock(PipelineSteps)`. `def deployer = new DeployPipeline(steps)`. `when: deployer.deploy("api", "v2.1")`. `then: 1 * steps.sh("kubectl apply -f deploy.yaml")`. Verifies the shared library calls the expected commands without actually running them.

Q45 Groovy pipeline optimization

The Simple Answer

Parallel stages: Run independent stages simultaneously. Test suites in parallel halve pipeline time.
Matrix builds: Test across multiple dimensions: `matrix { axes { axis { name "NODE"; values "16", "18", "20" }; axis { name "OS"; values "ubuntu", "alpine" } } }`. Generates 6 combinations automatically.
Stash/unstash: `stash(name: "build", includes: "dist/")` saves files. `unstash("build")` restores them on another agent. Avoids rebuilding on each agent. Workspace cleanup: `** Prevent disk fill. cleanWs()` in `post.cleanup`.

Q46 JSON basics - syntax, vs YAML

The Simple Answer

JSON (JavaScript Object Notation) is a lightweight data format. Objects (`{}`), arrays (`[]`), strings (`""`), numbers, booleans (`true/false`), null. No comments (unlike YAML). Strict syntax - every bracket, comma, and quote must be correct. Machine-friendly. Every programming language has a JSON parser.

JSON vs YAML

Feature	JSON	YAML
Readability	Lower (brackets, quotes)	Higher (indentation-based)
Comments	Not supported	Supported (#)
Data types	String, number, bool, null	Same + dates, timestamps
Strictness	Very strict	More flexible (gotchas)
File size	Larger (more syntax)	Smaller (less syntax)
Parsing speed	Faster	Slower
Best for	APIs, machine data	Human-edited configs

Q47 JSON Schema - validation

The Simple Answer

JSON Schema defines the structure and constraints of JSON data. Validate API payloads, configuration files, and pipeline parameters before processing them.

Example

`{"type": "object", "required": ["name", "port"], "properties": {"name": {"type": "string", "minLength": 1}, "port": {"type": "integer", "minimum": 1, "maximum": 65535}, "ssl": {"type": "boolean", "default": true}}`. Validation: the name must be a non-empty string, port must be 1-65535, ssl is optional (defaults to true). Invalid input is rejected before the script runs - preventing runtime errors.

Q48 jq command-line - filters, transformations

The Simple Answer

jq is the sed/awk for JSON. It filters, transforms, and extracts data from JSON on the command line. Essential for processing API responses in shell scripts and CI/CD pipelines.

Essential Filters

Extract field: `echo '{"name":"web","port":80}' | jq '.name'` returns "web". Array element: `echo '[1,2,3]' | jq '.[0]'` returns 1. Array iteration: `echo '[{"name":"a"}, {"name":"b"}]' | jq '.[].name'` returns "a" and "b". Select/filter: `jq '.[] | select(.status == "running")'` - filter array to running items only. Create new object: `jq '{id: .InstanceId, state: .State.Name}'` - reshape JSON.

Real-World Pipeline Usage

`aws ec2 describe-instances | jq -r '.Reservations[].Instances[] | select(.State.Name == "running") | .InstanceId'` - List running EC2 instance IDs. `kubectl get pods -o json | jq '.items[] | select(.status.phase != "Running") | .metadata.name'` - Find non-running pods.

Q49 jq advanced - select, map, reduce

The Simple Answer

map: `jq '.[[]] | {name: .metadata.name, status: .status.phase}'` - Transform each element. select with conditions: `jq '.[[]] | select(.cpu > 80 and .memory > 90)'` - Multiple conditions. group_by: `jq 'group_by(.region) | map({region: .[0].region, count: length})'` - Group and count. sort_by: `jq 'sort_by(.created) | reverse'` - Sort descending. reduce: `jq 'reduce .[] as $item (0; . + $item.cost)'` - Sum costs. @csv output: `jq -r '.[[]] | [.name, .status] | @csv'` - JSON to CSV.

Q50 HCL basics - blocks, attributes

The Simple Answer

HCL (HashiCorp Configuration Language) is used by Terraform, Packer, Vault, Consul, and Nomad. It combines the readability of YAML with the expressiveness of a programming language.

Syntax

Blocks: resource "aws_instance" "web" { ami = "ami-123"; instance_type = "t3.medium"; tags = { Name = "web-server" } }. Attributes: key = value. Strings in quotes. Numbers without quotes. Booleans: true/false. Lists: ["a", "b", "c"]. Maps: { key1 = "value1", key2 = "value2" }.

HCL vs JSON

Terraform accepts both HCL and JSON. HCL is for humans: resource "aws_instance" "web" { }. JSON equivalent: {"resource": {"aws_instance": {"web": {}}}} - more verbose, harder to read. Always use HCL for Terraform configs.

Q51 HCL expressions - variables, locals, functions

The Simple Answer

Variables: variable "region" { default = "us-east-1" }. Reference: var.region. Locals: locals { name_prefix = "\${var.project}-\${var.env}" }. Reference: local.name_prefix. Computed values that avoid repetition. Functions: length(var.subnets), lookup(var.amis, var.region), format("web-%02d", count.index). Conditionals: instance_type = var.env == "prod" ? "m5.xlarge" : "t3.medium". Loops: for_each = toset(var.instances), dynamic blocks for repeated nested blocks.

Q52 HCL vs JSON vs YAML

The Simple Answer

Feature	HCL	JSON	YAML
Comments	# and // and /* */	None	#
Variables	Built-in	None	None (Jinja2 in Ansible)
Functions	Built-in (50+)	None	None
Conditionals	Ternary operator	None	None
Loops	for_each, count, dynamic	None	None
Best for	Infrastructure code (Terraform)	API data, machine interchange	Human-edited configs

HCL is the most powerful for infrastructure definitions but is specific to HashiCorp tools. JSON is universal but verbose and not human-friendly for editing. YAML is the most human-readable but has gotchas and no built-in logic.

Q53 YAML validation - yamllint

The Simple Answer

yamllint checks YAML files for: syntax errors, indentation inconsistencies, line length, trailing spaces, and truthy values (the Norway problem). Install: `pip install yamllint`. Run: `yamllint config.yaml`. Configure in `.yamllint.yml`: customize rules per project. Integrate in: pre-commit hooks and CI pipelines. Every YAML file committed to the repository should pass yamllint.

Q54 JSON in CI/CD - parsing pipeline data

The Simple Answer

CI/CD pipelines constantly process JSON: API responses from cloud providers, test results, build metadata, deployment status. jq is the primary tool. In GitHub Actions: `echo '${{ toJson(github) }}' | jq '.actor'`. In scripts: `curl -s api.example.com/status | jq -r '.version'`. For complex processing, use Python: `import json, sys; data = json.load(sys.stdin); print(data["build"]["version"])`.

Q55 Config languages comparison - YAML vs JSON vs HCL vs TOML

The Simple Answer

YAML - Human-readable, indentation-based. Used by: K8s, Ansible, Docker Compose, GitHub Actions. Gotcha: boolean coercion, indentation errors. JSON - Machine-readable, strict syntax. Used by: APIs, package.json, tsconfig.json. No comments. HCL - Infrastructure-specific, built-in logic. Used by: Terraform, Packer, Vault. Most powerful for infra. TOML - Simple key-value, sections with [headers]. Used by: Cargo (Rust), pyproject.toml, Hugo. Clean and simple.

****The decision:**** Use what the tool requires. Kubernetes needs YAML. Terraform needs HCL. Node needs JSON. For your own tools: YAML if humans edit it frequently. JSON if machines generate and consume it. TOML for simple configuration files.

SECTION 3 - Senior Architecture (Q56–Q65)

Q56 Python infrastructure automation - custom providers, operators

The Simple Answer

Beyond scripting, Python builds infrastructure tools. Custom Terraform providers (using the Terraform Plugin Framework or CDK for Terraform in Python), Kubernetes operators (using kopf - Kubernetes Operator Pythonic Framework), and internal platforms (CLI tools that wrap terraform, kubectl, and cloud APIs into organization-specific workflows).

Q57 Python API development - FastAPI for internal tools

The Simple Answer

Internal DevOps APIs serve: webhook receivers (GitHub → trigger deployment), self-service portals (developers request environments), status dashboards (aggregate health from multiple services), and chatbot backends (Slack commands trigger automation).

FastAPI

Modern Python web framework. Async by default. Auto-generated OpenAPI docs. Type validation. from `fastapi import FastAPI; app = FastAPI(); @app.get("/health"); def health(): return {"status": "ok"}; @app.post("/deploy"); async def deploy(service: str, version: str): result = await run_deploy(service, version); return result. Deploy with: uvicorn app:app --host 0.0.0.0 --port 8000. Docker: FROM python:3.11-slim → pip install fastapi uvicorn → CMD uvicorn.`

Q58 Python data pipelines - logs, metrics, ETL

The Simple Answer

DevOps generates massive data: logs, metrics, events, audit trails. Python processes this data for: analysis (find error patterns in logs), alerting (detect anomalies in metrics), reporting (generate daily deployment summaries), and compliance (audit trail processing).

Pattern: Stream Processing

Use generators for memory efficiency. Read log lines → filter relevant events → parse structured data → aggregate → output. Tools: pandas for analysis (but memory-heavy), polars for faster processing, or pure Python generators for streaming large files.

Q59 Groovy to declarative migration

The Simple Answer

Migrating from scripted to declarative pipelines: map each `node{}` to `agent`, each `stage{}` stays, move conditional logic to `when{}` blocks, wrap complex Groovy in `script{}` blocks, and extract reusable logic into shared libraries. Declarative limitations: cannot define functions at top level (use shared libs), cannot use arbitrary Groovy outside `script{}` blocks.

Q60 Pipeline as code architecture

The Simple Answer

Monorepo Jenkinsfile: Each project has its own Jenkinsfile. Simple but duplicated. Changes to the pipeline pattern require updating 50 files.

Shared library + thin Jenkinsfile: Projects have a 5-line Jenkinsfile that loads the shared library and calls: `standardPipeline(language: "java", deployTarget: "kubernetes")`. All pipeline logic lives in the shared library. Change once, affect all. The recommended pattern for organizations with 10+ services.

Q61 YAML templating engines - ytt, Jsonnet, Cue

The Simple Answer

When plain YAML is not enough (too much copy-paste, no variables, no loops), templating engines add logic.

ytt (Carvel) - YAML-aware templating. Python-like Starlark language in YAML comments. Maintains valid YAML at all times. Used in Kubernetes (part of Carvel toolkit). Jsonnet - JSON superset with variables, functions, imports. Generates JSON/YAML. Used by Grafana for dashboard generation. More powerful than YAML but unfamiliar syntax. Cue - Type-safe configuration language. Validates AND generates configs. Catches errors at definition time, not deployment time. Growing adoption. Dhall - Functional configuration language. Mathematically guaranteed to terminate. Used by teams that need configuration correctness guarantees.

Q62 Config anti-patterns - YAML hell

The Simple Answer

Copy-paste configs - 50 Kubernetes manifests with 90% identical content. Fix: Helm charts, Kustomize overlays, or ytt templates. Environment drift - Dev config diverges from prod over months. Fix: generate all environments from the same template with env-specific values. Secrets in plaintext - Passwords in values.yaml committed to Git. Fix: Vault, Sealed Secrets, SOPS. Magic values - Undocumented numbers: replicas: 7. Why 7? Document: replicas: 7 # Based on load test: 7 replicas handle 10k RPS. No validation - Invalid YAML deployed to production. Fix: yamllint, kubeval/kubeconform, JSON Schema in CI.

Q63 Polyglot scripting - when which language

The Simple Answer

Bash - Glue scripts under 50 lines. Piping commands. Quick automation. System administration. Python - Complex automation (50+ lines). API interactions. Data processing. Cloud SDKs. Internal tools. Go - Performance-critical CLI tools (kubectl, terraform, docker are all Go). Cross-compiled single binary. When Python is too slow. Groovy - Jenkins pipelines only. Do not use for general scripting. TypeScript - CDK for Terraform, Pulumi, GitHub Actions (JavaScript runtime). Frontend-focused teams.

****The team decision:**** Standardize on 2 languages maximum for all automation. Most teams: Bash for simple tasks + Python for everything else. Add Go only if building tools that need distribution as single binaries.

Q64 Internal developer platforms - CLI tools

The Simple Answer

As DevOps matures, teams build internal tools: deploy (wraps kubectl + Helm + verification), infra (wraps Terraform with org-specific conventions), oncall (queries PagerDuty, shows runbooks, opens incident channels). Build with Python + click for CLI. Distribute via pip install (internal PyPI). Document with auto-generated --help.

Q65 Code quality for scripts

The Simple Answer

Automation code deserves the same quality standards as application code. Lint: ruff (Python), shellcheck (Bash), npm-groovy-lint (Groovy). Test: pytest with moto for AWS scripts, Spock for Groovy. CI: run lints and tests on every PR that modifies scripts. Review: code review for automation changes - a bad deploy script causes more damage than a bad API endpoint. Version: semantic versioning for shared libraries and internal tools.

SECTION 4 - Tricky Scenarios (Q66–Q80)

Q66 Python ModuleNotFoundError in CI

Step-by-Step

Step 1 - Check the virtual environment. CI creates a fresh environment each run. Is pip install -r requirements.txt running BEFORE the script? Is the correct Python version used (python3.11 vs python3.9)?

Step 2 - Check the requirements.txt. Is the module listed? Is the correct package name used (PyYAML not pyyaml, Pillow not PIL)?

Step 3 - Check the import path. Is the module a local file? CI checks out the code in a different directory than your laptop. Use relative imports or proper packaging.

Step 4 - Fix. Pin all dependencies in requirements.txt. Use a Dockerfile for CI (consistent environment). Or use poetry with poetry.lock for exact reproducibility.

Q67 Boto3 script hangs

Step-by-Step

Step 1 - Check region. Boto3 without region_name waits for metadata service timeout (on non-EC2 machines). Fix: always specify region: boto3.client("ec2", region_name="us-east-1") or set AWS_DEFAULT_REGION.

Step 2 - Check credentials. Boto3 credential chain tries: env vars → config file → instance profile → ECS task role. If no credentials found, it hangs waiting for the metadata service. Fix: set AWS_ACCESS_KEY_ID/AWS_SECRET_ACCESS_KEY or ensure the IAM role is attached.

Step 3 - Check network. Boto3 calls AWS API endpoints. If the network is restricted (no internet, no VPC endpoint), the call hangs at TCP connection. Fix: ensure network access to AWS APIs.

Q68 Python OOM processing 10GB log file

Step-by-Step

Step 1 - The problem. data = open("huge.log").read() loads 10 GB into memory. Python process killed by OOM killer.

Step 2 - Stream processing. with open("huge.log") as f: for line in f: if "ERROR" in line: process(line). Only one line in memory at a time. Memory usage: kilobytes instead of gigabytes.

Step 3 - For structured processing. Use generators: def parse_errors(path): with open(path) as f: for line in f: if "ERROR" in line: yield json.loads(line). Chain generators for multi-step processing without memory overhead.

Step 4 - For aggregation. Use collections.Counter for counting patterns. Counter updates incrementally - no need to hold all data in memory.

Q69 Jenkins Groovy sandbox violation

Step-by-Step

Step 1 - Read the error. "Scripts not permitted to use method java.io.File exists." The sandbox blocked a Java/Groovy method.

Step 2 - Replace with pipeline steps. `new File("path").exists()` → `fileExists("path")`. `new File("path").text` → `readFile("path")`. `"curl".execute()` → `sh(script: "curl ...", returnStdout: true)`.

Step 3 - If no pipeline step exists. Ask a Jenkins admin to approve the method in Manage Jenkins → In-process Script Approval. Or move the logic to a shared library with `@NonCPS` annotation.

Step 4 - For shared libraries. Code in `src/` classes runs outside the sandbox (trusted). Move complex logic there. `vars/` functions run inside the sandbox.

Q70 Jenkins shared library breaking after upgrade

Step-by-Step

Step 1 - Check the Jenkins changelog. Which pipeline steps changed? Deprecated methods removed? New sandbox restrictions?

Step 2 - Pin the library version. `@Library("my-lib@v2.1.0")` _ - Use a specific Git tag, not branch. This prevents automatic updates from breaking pipelines.

Step 3 - Test before upgrading. Run the shared library tests (Spock) against the new Jenkins version in a test instance. Fix incompatibilities before updating production Jenkins.

Q71 YAML indentation breaks K8s deployment

Step-by-Step

Step 1 - The error. `kubectl apply -f deploy.yaml`: "error parsing deploy.yaml: mapping values are not allowed in this context."

Step 2 - Find the indentation error. `yamllint deploy.yaml` shows the exact line. Common: mixing 2-space and 4-space indentation, tab character (invisible but rejected by YAML), or a key at the wrong nesting level.

Step 3 - Fix. Use an editor with YAML support (VS Code with YAML extension). Enable "show whitespace" to see tabs vs spaces. Use `yamllint` in pre-commit hooks to catch these before they reach `kubectl`.

Q72 YAML boolean coercion - yes becomes true

Step-by-Step

Step 1 - The symptom. A `ConfigMap` value `country: NO` is read by the application as `false` instead of the string `"NO"`. Or: `feature_flag: yes` becomes the boolean `true`.

Step 2 - Root cause. YAML 1.1 coerces yes/no/on/off/y/n to booleans. The YAML parser converts the string before it reaches your application.

Step 3 - Fix. Quote the value: country: "NO", feature_flag: "yes". Quoted values are always strings.

Step 4 - Prevent. Add a yamllint rule: truthy: {check-keys: true} alerts on unquoted truthy values. Use kubeconform to validate K8s manifests against the schema - it catches type mismatches.

Q73 jq returns empty on nested JSON

Step-by-Step

Step 1 - The problem. `aws ec2 describe-instances | jq '.Instances[].InstanceId'` returns nothing. The data exists but jq cannot find it.

Step 2 - Debug. Pipe through `jq '.'` first to see the full structure. The instances are likely nested: `.Reservations[].Instances[].InstanceId` - there is an extra level of nesting.

Step 3 - Fix. Use the correct path. Or use recursive descent: `jq '.. | .InstanceId? // empty'` finds InstanceId at any depth. For debugging: `jq 'keys'` shows top-level keys, `jq '.[0] | keys'` shows second-level keys.

Q74 Python requests timeout - retry logic

Step-by-Step

Step 1 - The problem. `requests.get(url)` hangs indefinitely because the server is slow or unresponsive. No timeout was set.

Step 2 - Add timeout. `requests.get(url, timeout=(5, 30))` - 5 seconds for connection, 30 seconds for response.

Step 3 - Add retry logic. `from urllib3.util.retry import Retry; from requests.adapters import HTTPAdapter. retry = Retry(total=3, backoff_factor=1, status_forcelist=[500, 502, 503, 504]). session = requests.Session(); session.mount("https://", HTTPAdapter(max_retries=retry)). response = session.get(url, timeout=30). Retries 3 times with exponential backoff (1s, 2s, 4s) for server errors.`

Q75 HCL for_each unexpected resource names

Step-by-Step

Step 1 - The problem. `for_each = var.subnets` creates resources like `aws_subnet.main["10.0.1.0/24"]`. The CIDR is the map key - ugly and problematic if the CIDR changes (Terraform destroys and recreates the resource).

Step 2 - Fix. Use meaningful stable keys: `for_each = { public-a = { cidr = "10.0.1.0/24", az = "us-east-1a" }, public-b = { ... } }`. Resources become: `aws_subnet.main["public-a"]` - stable, readable names. Changing the CIDR does not force recreation.

Q76 Groovy CPS transformation error

Step-by-Step

Step 1 - The error. "java.io.NotSerializableException" or "CPS-transformed closure." Jenkins transforms Groovy code for pipeline durability (CPS - Continuation Passing Style). Some Groovy features (closures with certain methods, Java streams) are not CPS-compatible.

Step 2 - Fix. Add `@NonCPS` annotation to methods that do not need CPS: `@NonCPS def parseJson(text) { new JsonSlurper().parseText(text) }`. Move non-serializable operations to `@NonCPS` methods. Keep CPS methods (those that call pipeline steps like `sh`, `echo`) without the annotation.

Q77 Python subprocess hangs - deadlock

Step-by-Step

Step 1 - The problem. `subprocess.Popen` with `PIPE` for `stdout` and `stderr`. The child process writes more output than the pipe buffer (64 KB). The pipe fills up. The child blocks on write. The parent blocks on `wait()`. Deadlock.

Step 2 - Fix. Use `subprocess.run()` with `capture_output=True` (handles buffering internally). Or use `communicate()`: `proc = Popen(cmd, stdout=PIPE, stderr=PIPE); stdout, stderr = proc.communicate()`. `communicate()` reads both pipes simultaneously, preventing deadlock.

Q78 YAML merge key deprecated

Step-by-Step

Step 1 - The issue. The `<<` merge key is a YAML 1.1 extension, not part of the YAML 1.2 spec. Some modern parsers reject it or warn about it. Future tools may not support it.

Step 2 - Alternatives. Use tool-specific features: Kustomize overlays (patch instead of merge), Helm values (inheritance through defaults), Ansible variables (`group_vars` hierarchy), or YAML templating engines (`ytt`, `Jsonnet` - proper variables and functions).

Step 3 - Migration. Replace `<<` anchors with explicit values. Or move to a templating engine that handles reuse properly. For GitHub Actions: use composite actions or reusable workflows instead of YAML anchors.

Q79 Write Python: auto-remediate AWS SG violations

The Design

Requirements: Monitor security groups. Detect rules allowing 0.0.0.0/0 on sensitive ports (22, 3389). Automatically remove the violating rule. Send a Slack notification.

Architecture

Lambda function triggered by AWS Config rule or EventBridge. Receives the event (security group change). Checks the new rules against policy (no 0.0.0.0/0 on port 22). If violation found: `ec2.revoke_security_group_ingress()` removes the rule. Sends Slack notification with: who created the rule, which SG, which port, and the remediation action taken. Logs everything for audit trail.

Key Code Patterns

Boto3 for EC2 operations. requests for Slack webhook. Structured logging for audit. Error handling with retry for API calls. Environment variables for configuration (Slack URL, excluded SGs). pytest with moto for testing.

Q80 Write Groovy: reusable deployment pipeline

The Design

Shared library step: vars/standardDeploy.groovy. Takes parameters: service name, Docker image, Kubernetes namespace, Helm chart path, values file. Stages: Lint (helm lint) → Build image → Push to registry → Deploy (helm upgrade --install --atomic) → Smoke test (curl health endpoint) → Notify (Slack).

Pipeline Usage

@Library("platform-lib") _. standardDeploy(service: "payment-api", image: "registry.example.com/payment-api", namespace: "production", chart: "charts/payment-api", values: "values/production.yaml"). One line in the Jenkinsfile. All 50 microservices use the same shared library step. Change the deployment process once - all services updated.

Key Patterns

Default values for optional parameters. Error handling in each stage (block/rescue in deploy stage). Timeout for the entire pipeline. Input step for production approval. Configurable Slack channel per team. Version-pinned library reference.

Interview Tips

For Junior Roles

Focus on Q1-Q25. Write Python scripts: read a file, parse JSON, make API calls, process command output. Know basic Groovy for Jenkins. Write YAML without syntax errors. Know the difference between Python and Bash use cases.

For Mid-Level Roles

Q1-Q55. Be ready to write: a Python script with Boto3 (list resources, automate operations), a Jenkins pipeline with shared library calls, and a jq command to extract data from nested JSON. Know async, decorators, and context managers. Understand Pact contract testing and Testcontainers.

For Senior Roles

All 80 questions. Design internal CLI tools, explain polyglot scripting strategy, discuss YAML templating alternatives, and demonstrate Python packaging. Be ready to write a complete automation script (Q79) or design a shared library architecture (Q80).

Quick Reference Cheat Sheet

Topic	Key Takeaway
Python vs Bash	Bash for under 50 lines and piping commands. Python for everything else.
Python data types	Dicts are the workhorse. Lists for collections. f-strings for formatting.
Error handling	try/except with specific exceptions. Always set timeout on requests.
Boto3	Client (low-level) vs Resource (high-level). Always use paginators. Always set region.
Generators	yield for memory-efficient processing. Stream 10 GB files in kilobytes of RAM.
Decorators	@retry for automatic retry logic. @timer for performance monitoring.
async	asyncio.gather() for concurrent I/O. 100 health checks in 2 seconds vs 200 seconds.
Groovy	Jenkins pipeline language. Closures are the foundation. def and dynamic typing.
Scripted vs Declarative	Declarative for new pipelines. script {} blocks for Groovy power when needed.
Shared libraries	vars/ for pipeline steps. src/ for classes. Pin versions with @Library("lib@v2.1").
CPS errors	@NonCPS for non-serializable operations. Keep pipeline steps in CPS methods.
YAML	2 spaces, never tabs. Quote ambiguous values. yamllint in pre-commit.
YAML gotchas	"NO" becomes false. 0644 becomes 420. 12:30 becomes 750. Quote everything suspect.
YAML anchors	DRY within one file. Limited and being deprecated. Use tool-specific alternatives.
JSON/jq	jq is sed for JSON. '.field', '[][.field]', 'select(.x == "y")'. Essential for CI/CD.
HCL	Terraform's language. Variables, functions, loops built in. Use stable keys in for_each.
Config comparison	YAML for humans. JSON for machines. HCL for infrastructure. TOML for simple config.
Script quality	Lint (ruff/shellcheck) + test (pytest/moto) + CI on every PR. Scripts deserve quality too.

